

WEEK 3: SELLER AND INVENTORY MANAGEMENT SYSTEM

Scenario:

ArtHub is an online art marketplace where artworks are supplied by multiple sellers. The system should maintain seller details, track the artworks supplied by each seller, and monitor inventory levels to ensure the availability of artworks.

The Seller and Inventory Management System helps ArtHub maintain organized information about sellers and their artwork stock. It also helps identify available artworks, out-of-stock artworks, and overall inventory levels.

1. Create Seller and Inventory Tables

Seller Table:

The **Seller** table is created to store the details of sellers who supply artworks to ArtHub.

The table contains the following details:

- **Seller_ID** – Unique identification number of the seller.
- **Seller_Name** – Name of the seller.
- **Email** – Email address of the seller.
- **Phone** – Contact number of the seller.
- **Address** – Address of the seller.

Inventory Table:

The **Inventory** table is created to maintain the stock details of artworks available in ArtHub.

The table contains the following details:

- **Inventory_ID** – Unique identification number for the inventory record.
- **Artwork_ID** – Identifies the artwork associated with the inventory.
- **Seller_ID** – Identifies the seller supplying the artwork.
- **Stock_Quantity** – Number of artworks currently available.
- **Stock_Status** – Shows whether the artwork is available or out of stock.
- **Last_Updated** – Stores the date when the inventory information was last updated.

The two tables are created separately so that seller information and inventory information can be maintained in an organized manner.

```
SQL> CREATE TABLE Seller (Seller_ID int , Seller_Name VARCHAR2(50), Email VARCHAR2(100), Phone NUMBER, Address VARCHAR2(100));  
Table created.  
  
SQL> create table inventory(Inventory_ID varchar(10),artwork_ID varchar(10),seller_ID int,stock_Quantity number, stock_status varchar(10),last_updated DATE)  
;  
Table created.
```

2. Establish Relationships

After creating the Seller and Inventory tables, the relationships between the tables are identified.

Seller and Product

One seller can supply multiple artworks to ArtHub.

For example, a seller may supply different paintings, sketches, digital artworks, or sculptures.

Therefore:

One Seller → Many Artworks

Relationship:

Seller → Product is One-to-Many

The Seller_ID is used to identify the seller associated with the artwork information.

Product and Inventory

Each artwork has inventory information that is used to track its available stock.

The Inventory table stores the stock quantity and stock status of the artwork.

Therefore:

One Product → One Inventory Record

Relationship:

Product → Inventory is One-to-One

The Artwork_ID is used to connect the artwork with its inventory information.

3. Define Keys and Constraints

After identifying the relationships, suitable keys and constraints are applied to the tables.

Primary Key

A **Primary Key** is a column that uniquely identifies each row or record in a table.

In the ArtHub system:

- **Seller_ID** is the Primary Key of the Seller table.
- **Inventory_ID** is the Primary Key of the Inventory table.

This ensures that every seller and inventory record has a unique identification value.

```
SQL> alter table seller add constraint pk_seller primary key(seller_ID);
Table altered.

SQL> alter table inventory add constraint pk_inv primary key(Inventory_ID);
Table altered.
```

Foreign Key

A **Foreign Key** is used to establish a relationship between two tables.

In the Inventory table:

- **Artwork_ID** acts as a Foreign Key referring to the Artwork_ID in the ArtProduct table.
- **Seller_ID** acts as a Foreign Key referring to the Seller_ID in the Seller table.

This connects the inventory records with the corresponding artworks and sellers.

```
SQL> alter table inventory add constraint fk_artwork foreign key(artwork_ID) references artproduct (artwork_ID);
Table altered.
SQL> alter table inventory add constraint fk_seller foreign key(seller_ID) references seller (seller_ID);
Table altered.
```

UNIQUE Constraint

The **UNIQUE** constraint ensures that a particular value cannot be repeated in a column.

In the Seller table, the Email column is given a UNIQUE constraint.

Therefore, two different sellers cannot have the same email address.

```
SQL> alter table seller add constraint un_mail unique(Email);
Table altered.
```

NOT NULL Constraint

The **NOT NULL** constraint ensures that a column cannot contain an empty or NULL value.

The Email column in the Seller table is made NOT NULL because every seller should have an email address.

```
SQL> alter table seller modify Email NOT NULL;
Table altered.
```

CHECK Constraint

The **CHECK** constraint is used to ensure that the values entered into a column satisfy a particular condition.

In the Inventory table, the Stock_Quantity is checked to ensure that it is not negative.

This prevents invalid values such as negative stock quantities from being entered.

```
SQL> alter table inventory add constraint ck_quan check (stock_quantity >=0)
Table altered.
```

4. Insert Values

After creating the tables and applying the required constraints, values are inserted into the Seller and Inventory tables.

Insert Values to Seller Table

```
SQL> INSERT INTO Seller VALUES (1, 'Art House', 'arthouse@gmail.com', 9876543210, 'Chennai');
1 row created.

SQL> INSERT INTO Seller VALUES (2, 'Creative Corner', 'creativecorner@gmail.com', 9876543211, 'Bangalore');
1 row created.

SQL> INSERT INTO Seller VALUES (3, 'Sketch Studio', 'sketchstudio@gmail.com', 9876543212, 'Mumbai');
1 row created.

SQL> INSERT INTO Seller VALUES (4, 'Art World', 'artworld@gmail.com', 9876543213, 'Delhi');
1 row created.

SQL> INSERT INTO Seller VALUES (5, 'Digital Canvas', 'digitalcanvas@gmail.com', 9876543214, 'Hyderabad');
1 row created.

SQL> INSERT INTO Seller VALUES (6, 'Modern Arts', 'modernarts@gmail.com', 9876543215, 'Pune');
1 row created.

SQL> INSERT INTO Seller VALUES (7, 'Photo Arts', 'photoarts@gmail.com', 9876543216, 'Kochi');
1 row created.

SQL> INSERT INTO Seller VALUES (8, 'Sculpture House', 'sculpturehouse@gmail.com', 9876543217, 'Chennai');
1 row created.

SQL> INSERT INTO Seller VALUES (9, 'Abstract Studio', 'abstractstudio@gmail.com', 9876543218, 'Bangalore');
1 row created.

SQL> SELECT s.Seller_Name, COUNT(i.Artwork_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Quantity > 5 GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS
Abstract Studio	1
Sketch Studio	1
Art House	1

SELLER_NAME	NUMBER_OF_PRODUCTS
Digital Canvas	1
Photo Arts	1
Craft Gallery	1

6 rows selected.

Insert Values to Inventory Table

```
SQL> INSERT INTO Inventory VALUES ('INV01', '101', 1, 6, 'Available', TO_DATE('01-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV02', '102', 2, 0, 'Out', TO_DATE('03-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV03', '103', 3, 18, 'Available', TO_DATE('05-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV04', '104', 4, 4, 'Available', TO_DATE('07-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV05', '105', 5, 25, 'Available', TO_DATE('09-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV06', '106', 6, 0, 'Out', TO_DATE('11-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV07', '107', 7, 13, 'Available', TO_DATE('13-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV08', '108', 8, 3, 'Available', TO_DATE('15-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV09', '109', 9, 21, 'Available', TO_DATE('17-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Inventory VALUES ('INV10', '110', 10, 9, 'Available', TO_DATE('19-09-2026','DD-MM-YYYY'));
```

```
SQL> select * from inventory;
```

INVENTORY_	ARTWORK_ID	SELLER_ID	STOCK_QUANTITY	STOCK_STAT	LAST_UPDATED
INV01	101	1	6	Available	01-SEP-26
INV02	102	2	0	Out	03-SEP-26
INV03	103	3	18	Available	05-SEP-26
INV04	104	4	4	Available	07-SEP-26
INV05	105	5	25	Available	09-SEP-26
INV06	106	6	0	Out	11-SEP-26
INV07	107	7	13	Available	13-SEP-26
INV08	108	8	3	Available	15-SEP-26
INV09	109	9	21	Available	17-SEP-26
INV10	110	10	9	Available	19-SEP-26

10 rows selected.

5. Generate Seller and Inventory Reports

After inserting the records, different reports are generated to analyze seller and inventory information. Three reports are generated:

1. **Seller-wise Product Report**
2. **Stock Availability Report**
3. **Inventory Analysis**

Report 1: Seller-wise Product Report

The **Seller-wise Product Report** is used to display the artwork supply information for each seller.

Display:

- Seller Name, Number of Products Supplied, Available Stock

The report combines information from the Seller and Inventory tables to identify the number of artworks supplied by each seller and the available stock.

A condition is used to display sellers associated with inventory records having stock quantity greater than 5.

This report helps ArtHub understand the number of artworks supplied by different sellers and the amount of stock currently available from them.

```
SQL> SELECT s.Seller_Name, COUNT(i.Artwork_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Quantity > 5 GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Abstract Studio	1	21
Sketch Studio	1	18
Art House	1	6
Digital Canvas	1	25
Photo Arts	1	13
Craft Gallery	1	9

6 rows selected.

Report 2: Stock Availability Report

The **Stock Availability Report** is used to display the current availability of artworks in ArtHub.

Display:

- Product Name
- Stock Quantity
- Stock Status

The product name is obtained from the ArtProduct table, while stock quantity and stock status are obtained from the Inventory table.

The report helps ArtHub identify the current stock level of each artwork and whether the artwork is available or out of stock.

This information is useful for monitoring product availability on the marketplace.

```
SQL> SELECT a.Artwork_Name, i.Stock_Quantity, i.Stock_Status FROM ArtProduct a JOIN Inventory i ON a.Artwork_ID = i.Artwork_ID WHERE i
.Stock_Quantity >= 0;

ARTWORK_NAME
-----
STOCK_QUANTITY STOCK_STAT
-----
Sunset Over Hills
           6 Available

Colors of Nature
           0 Out

Pencil Portrait
          18 Available

ARTWORK_NAME
-----
STOCK_QUANTITY STOCK_STAT
-----
Charcoal Dreams
           4 Available

Digital Galaxy
          25 Available

Future City
           0 Out

ARTWORK_NAME
-----
STOCK_QUANTITY STOCK_STAT
-----
```

Report 3: Inventory Analysis

The **Inventory Analysis** report provides an overall analysis of the inventory maintained by ArtHub.

Find:

- Total Products Available
- Products Out of Stock
- Highest Stocked Products
- Average Inventory Quantity

Total Products Available

This analysis identifies the total number of artworks that currently have stock available.

Products with a stock quantity greater than zero are considered available.

This helps ArtHub understand how many artworks are currently available for customers.

```
SQL> SELECT COUNT(*) AS Total_Products_Available FROM Inventory WHERE Stock_Quantity > 0;

TOTAL_PRODUCTS_AVAILABLE
-----
                        8
```

Products Out of Stock

This analysis identifies the number of artworks that have no available stock.

Products with a stock quantity of zero are considered out of stock.

This information helps ArtHub identify artworks that may require restocking.

```
SQL> SELECT COUNT(*) AS Products_Out_of_Stock FROM Inventory WHERE Stock_Quantity = 0;

PRODUCTS_OUT_OF_STOCK
-----
                        2
```

Highest Stocked Products

This analysis identifies the artwork with the highest stock quantity in the inventory.

It helps ArtHub determine which artwork currently has the largest number of units available.

```
SQL> SELECT a.Artwork_Name, i.Stock_Quantity FROM ArtProduct a JOIN Inventory i ON a.Artwork_ID = i.Artwork_ID WHERE i.Stock_Quantity
= (SELECT MAX(Stock_Quantity) FROM Inventory);

ARTWORK_NAME
-----
STOCK_QUANTITY
-----
Digital Galaxy
                25
```

Average Inventory Quantity

This analysis calculates the average inventory quantity of the artworks maintained by ArtHub.

The average inventory quantity provides an overall view of the stock level maintained across the artworks.

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory_Quantity FROM Inventory WHERE Stock_Quantity >= 0;

AVERAGE_INVENTORY_QUANTITY
-----
                        9.9
```